Chapter 3 Solutions

CH03-Q01

`0x7f + 0x01` produces `0x80`, signed −128. Its mathematical value 128 exceeds 127, so signed overflow occurs.

`0x80 + 0xff` is mathematically −128−1=−129. Its low bits are `0x7f`, signed 127. Signed overflow occurs.

For `0x80 - 0x01`, compute `0x80 + 0xfe + 1`. The low result is again `0x7f`, with signed overflow. Using `0xff` as the complement of `0x01` would be an incorrect subtraction trace.

Signed overflow is $c_{\text{into MSB}} \oplus c_{\text{out of MSB}}$. Unsigned overflow uses only the outgoing carry.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

CH03-Q02

Round	Old Q0	Add result C:A	A after shift	Q after shift
1	1	`0:1101`	`0110`	`1101`
2	1	`1:0011`	`1001`	`1110`
3	0	`0:1001`	`0100`	`1111`
4	1	`1:0001`	`1000`	`1111`

Round 2 shifts full input `1:0011:1101` to low eight bits `10011110`. The carry enters A; it is not accumulated a second time. The final concatenation is `(A,Q)=10001111`, representing $143 = 13 \times 11$.

Fix the initial multiplier $q < 2^n$, and define $R_k = 2^n A_k + Q_k$ after round k. The exact invariant is

$$R_k = \left\lfloor \frac{q}{2^k} \right\rfloor + M(q \bmod 2^k)2^{n-k}, \qquad 0 \le k \le n.$$

At $k = 0$, $A_0 = 0, Q_0 = q$, so it holds. When $k < n$, the second term is even. The current low bit is therefore original multiplier bit q_k. The conditional addition produces full combined value $R_k + q_k M 2^n$, including the carry. Shifting gives

$$R_{k+1} = \left\lfloor \frac{R_k + q_k M 2^n}{2} \right\rfloor = \left\lfloor \frac{q}{2^{k+1}} \right\rfloor + M(q \bmod 2^{k+1})2^{n-k-1}.$$

The equality uses $q \bmod 2^{k+1} = (q \bmod 2^k) + q_k 2^k$, completing induction. At $k = n$, $q < 2^n$ makes the first term zero, so $R_n = Mq$. A contains the high half and Q the low half. Carry C enters A during the shift and is not lost. The processed old Q0 leaves; old Q1 becomes the next decision bit.

For this case R0..R4 are 11,109,158,79,143, matching the table.

$$T_{\text{serial}} = 3 + 32(1 + 1) + 2 = 69 \text{ cycles}.$$

An initiation interval of four cycles gives $1/4 = 0.25$ multiplication/cycle while a result may still take 69 cycles. Overlap requires separate partial-product/control state, collision handling, and contexts or stage resources for concurrent operations; merely duplicating the ALU is insufficient.

CH03-Q03

Round	Shifted A,Q	Trial A−M	State after decision
1	`00001,1010`	−4	Restore, `00001,1010`
2	`00011,0100`	−2	Restore, `00011,0100`
3	`00110,1000`	1	Keep and set bit, `00001,1001`
4	`00011,0010`	−2	Restore, `00011,0010`

Thus $Q = 2, R = 3$. Check $13 = 2 \times 5 + 3$ and $0 \leq 3 < 5$.

The signed boundary is $-2147483648 / -1 = 2147483648$, above the signed maximum. In traditional MIPS32, `div` does not automatically provide an overflow trap with a dependable quotient. Software must check the operands. Other architectures may define a trap or another result.

Independent 11/3 trace and general proof

k	New bit	P_k	T	T−3	q_k	A_k	Whole Q (4 bits)
0	—	0	—	—	0	0	`1011`
1	1	1	1	−2	0	1	`0110`
2	0	2	2	−1	0	2	`1100`
3	1	5	5	2	1	2	`1001`
4	1	11	5	2	3	2	`0011`

At round 3, q_k=1 but the entire Q equals 9 because its high bit remains unread input. Only after round 4 does Q=q_4=3. Check 11=3 • 3+2 and 0≤2<3.

Generally let T=2A_k+d and b=1 exactly when T≥M. Then $P_{k+1} = 2P_k + d = (2q_k + b)M + (T - bM)$. Since A_k<M, T<2M, so the new remainder T−bM remains in [0,M). Initialization is zero; induction through all n input bits proves the specification. The trace alone would not prove it for arbitrary inputs.

Truncation toward zero gives Q=−3,R=−2 for −11/3, and Q=3,R=−2 for −11/(−3). Check −11=(−3) • 3−2 and −11=3 • (−3)−2 with |R|<3. Floor division for −11/3 instead gives Q=−4,R=1, satisfying −11=(−4) • 3+1. Changing the rounding convention changes both quotient and remainder.

CH03-Q04

$12.5 = 1100.1_2 = 1.1001_2 \times 2^3$. The sign is 1, stored exponent is $3 + 127 = 130 = 10000010_2$, and fraction is `10010000000000000000000`.

The complete encoding is `1 | 10000010 | 10010000000000000000000 = 0xc1480000`.

For `0x3f000000`, sign is 0, exponent is 126, and fraction is zero. Value is $1.0 \times 2^{-1} = 0.5$.

The smallest normal has exponent field `00000001` and zero fraction, hence 2^{-126}. The smallest subnormal has zero exponent field and only fraction bit 0 set, hence $2^{-126}2^{-23} = 2^{-149}$.

`0x7f800000` is positive infinity. `0x7fc00001` has all-ones exponent and a nonzero fraction with the quiet bit set, so it is a quiet NaN in this binary32 convention.

CH03-Q05

Binary32 spacing above 1 is 2^{-23}. The candidates are 1 and $1 + 2^{-23}$. The exact sum lies halfway. Candidate 1 has an even low retained significand bit, so nearest-even returns 1, encoding `0x3f800000`.

The addition is inexact. Its flag records that rounding occurred, not the numerical error 2^{-24}. Trap enablement is a separate control.

A wider intermediate can retain $1 + 2^{-24}$. Subtracting 1 before the final conversion then gives 2^{-24}, exactly representable in binary32. Rounding the initial addition to binary32 first instead gives zero after subtraction.

CH03-Q06

$1.5 = 1.1_2 \times 2^0$, while $2^{-126} = 1.0_2 \times 2^{-126}$. Their significand product is 1.1_2, and their unbiased exponent sum is −126. The product is already normalized, so no left shift is needed.

Stored exponents contain biases. Adding them includes two biases; the result needs one. General multiplication may also require a one-bit right normalization and exponent increment, followed by another adjustment if rounding carries. Special and subnormal cases require separate rules.

CH03-Q07

> Spacing above 2^{24} is 2. Adding 1 reaches a midpoint between 2^{24} and $2^{24}+2$. Nearest-even chooses 2^{24}. Therefore $(x+y)-x=0$.
>
> The other grouping computes $x-x=0$, then adds 1, giving 1. The first rounding has lost the unit before cancellation can recover it.
>
> Under the specified wider-intermediate assumption, $(1+2^{-24})-1=2^{-24}$. With binary32 rounding after each operation it is zero. This conclusion depends on retained precision and control settings, not merely on an x87 label.

CH03-Q08

> Binary latency is $3+32+2=37$ cycles. Radix-4 latency is $6+16(1.5)=30$ cycle-equivalents. The fractional per-iteration model is an average or abstract timing model, not an assertion that a synchronous stage ends halfway through an ordinary clock cycle.
>
> By the separate acceptance-interval assumption, both throughputs are $1/8=0.125$ operations/cycle, or 125 operations per 1000 cycles after filling. No clock frequency is supplied, so operations per second cannot be calculated.
>
> Latency matters for dependent chains and critical paths. Throughput matters when enough independent operations keep resources occupied. A single ordinary iterative unit cannot sustain an eight-cycle interval with a 37-cycle operation without additional overlap or replication.
>
> For a divider, digit selection, normalization, correction, sign handling, zero-divisor checking, and remainder formation remain. Per-iteration timing can differ. Early termination can make costs input-dependent, in which case average time requires an operand distribution.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

CH03-Q09

At 1.0, the retained significand is `1.00000000000000000000000`. Adding 2^{-24} leaves GRS=`100`. This is a tie between 1 and $1 + 2^{-23}$; retained LSB zero selects 1, bits `0x3f800000`, with inexact set.

$3 \cdot 2^{-25} = 2^{-24} + 2^{-25}$ gives GRS=`110`. It exceeds half a retained unit, so rounding increments the fraction LSB and yields $1 + 2^{-23}$.

2^{-151} is below half the least positive subnormal 2^{-149}, so it rounds to +0. 2^{-150} is exactly halfway between +0 and 2^{-149}, selecting even +0, bits `0x00000000`. Both are tiny and inexact and set underflow and inexact.

Maximum finite is `0x7f7fffff`. Adding it to itself exceeds the range; default RNE returns `0x7f800000`, +infinity, with overflow and inexact. `0*infinity` is invalid and defaults to quiet NaN with invalid set, rather than zero.

CH03-Q10

The encodings are `a=b=0x3f800001`, with exponent 127 and fraction LSB one, and `c=0xbf800002`.

$ab = (1 + 2^{-23})^2 = 1 + 2^{-22} + 2^{-46}$.

At exponent zero the binary32 spacing is 2^{-23}. The residual 2^{-46} is far below half a ULP, so the separate multiply rounds to $1 + 2^{-22}$. Adding its exact negative gives +0, bits `0x00000000`. The addition cannot recover bits discarded by the multiply.

The fused exact intermediate is

$(1 + 2^{-22} + 2^{-46}) - (1 + 2^{-22}) = 2^{-46}$.

This is a normal binary32 value with biased exponent $127 - 46 = 81 = 0x51$, zero sign and fraction, hence `0x28800000`. A single rounding preserves a value lost by the separate product rounding.

At $x = 2^{24}$, ULP=2. `x+1` is a midpoint and rounds to even x, so `(x+1)-x=0`. Computing `x-x` first gives exact zero, then adding 1 gives 1. The different first rounding point causes the discrepancy. Each binary32 operation retains 24 significand bits; real associativity does not bypass these rounding operations.

CH03-Q11

1. Align $a=1.000_2\times2^0$ and $b=1.001_2\times2^{-4}=0.0001001_2\times2^0$. The sum is 1.0001001_2. Retain 1.000 with L=0,G=1,R=0; tail 01 gives S=1. Candidates 1 and 9/8 have midpoint 17/16=136/128. The exact sum 137/128 lies above it, selecting 9/8 with absolute error 7/128. Ignoring S creates a false GRS=100 tie and returns 1, with error 9/128 exceeding half-ULP=1/16.
2. Subtraction gives $0.001_2\times2^{-1}$. The left shifts give $0.010_2\times2^{-2}$, then $0.100_2\times2^{-3}$, then $1.000_2\times2^{-4}$. Each equals 1/16. No nonzero bits are discarded, so inexact=0.
3. The candidates for 1.1111_2 are 1.111_2 (significand integer 15, odd) and 10.000_2 (integer 16, even). With L=1 and GRS=100, choose the latter, renormalizing to $1.000_2\times2^1$. For -1.0011_2, magnitude candidates are 1.001_2=9/8 (integer 9, odd) and 1.010_2=5/4 (integer 10, even). Exact magnitude 19/16 is halfway; choose −5/4. Incrementing the magnitude makes this negative result more negative, rather than rounding toward $+\infty$. Both cases are inexact.

CH03-Q12

(1) The exact quotient $2^{-126}/2=2^{-127}=(2^{22})\delta$ has fraction integer $F=2^{22}$ and exponent field 0, giving 0x00400000. It is exact: underflow=0,inexact=0.
For δ/2 the candidates are 0 (F=0, even) and δ (F=1, odd). The tie selects +0, bits 0x00000000. The result is tiny and inexact, setting underflow=1,inexact=1.
For 3δ/2 the candidates are δ (F=1, odd) and 2δ (F=2, even). Select 2δ, bits 0x00000002. The result differs from the exact value, so again underflow=1,inexact=1.
(2) Smallest normal is $2^{23}\delta$ and largest subnormal is $(2^{23}-1)\delta$. Their difference is exactly δ, bits 0x00000001. Both flags remain 0. Small magnitude alone does not determine the underflow flag.
(3) With exponent field 0, the value is $(F/2^{23})\cdot2^{-126}=F\delta$. Each unit increase in F adds δ. The last subnormal and first normal also differ by δ, with no enlarged gap. Fixed absolute spacing does not mean fixed relative precision: the same δ is a larger fraction of values nearer zero.

CH03-Q13

Sign-extend $M = -8$ as `11000` and start with `00000 1101 0`:

Round	Old Q_0q_{-1}	A action	After add/sub: A Q q_{-1}	After shift: A Q q_{-1}
1	`10`	−M	`01000 1101 0`	`00100 0110 1`
2	`01`	+M	`11100 0110 1`	`11110 0011 0`
3	`10`	−M	`00110 0011 0`	`00011 0001 1`
4	`11`	0	`00011 0001 1`	`00001 1000 1`

Discard the history bit: $(A, Q) =$`00001 1000`. Remove the redundant sign bit to get `00011000`, or 24. The multiplier recodes as $-1 + 2 - 4 = -3$, giving $8 - 16 + 32 = 24$.

CH03-Q13 (continued)

Why the four-bit path fails

With only four bits, the first round requires $0 - (-8) = 8$, but `1000` means -8. The incorrect trace is:

Round	Old Q_0q_{-1}	A action	After add/sub: A Q q_{-1}	After shift: A Q q_{-1}
1	`10`	−M	`1000 1101 0`	`1100 0110 1`
2	`01`	+M	`0100 0110 1`	`0010 0011 0`
3	`10`	−M	`1010 0011 0`	`1101 0001 1`
4	`11`	0	`1101 0001 1`	`1110 1000 1`

Final `11101000` means -24, not 24. Signed overflow occurs in the first round; later arithmetic shifts cannot restore the sign information already lost.

CH03-Q13 (continued)

Arithmetic operations and clock cycles

For `0101`, low-to-high digits are $(-1, +1, -1, +1)$: $-1 + 2 - 4 + 8 = 5$, requiring four arithmetic operations. Unsigned shift-add sees only two ones and needs two additions. Four rounds do not establish four cycles without timing rules for arithmetic, shifting and control.

CH03-Q13 (continued)

An independent radix-4 example

Six-bit -13 is `110011`. Append history0. The overlapping groups are `(q₁q₀q₋₁)=110`, `(q₃q₂q₁)=001`, `(q₅q₄q₃)=110`. Digits $(-1, +1, -1)$ give $-13 = -1 + 4 - 16$.

j	Digit	Partial product	Twelve-bit representation
0	−1	+5	`000000000101`
1	+1	−20	`111111101100`
2	−1	+80	`000001010000`

The sum is $5 - 20 + 80 = 65$, or `000001000001`. Substitute $d_{2j} = q_{2j-1} - q_{2j}$ and $d_{2j+1} = q_{2j} - q_{2j+1}$ into $c_j = d_{2j} + 2d_{2j+1}$. This yields $c_j = q_{2j-1} + q_{2j} - 2q_{2j+1}$, which determines all eight table entries.

CH03-Q14

CH03-Q14 (continued)

1. Check the total at each level

Row	Eight bits	Unsigned value
P_0	00001011	11
P_1	00010110	22
P_2	00101100	44
P_3	01011000	88
S_1=P_0 XOR P_1 XOR P_2	00110001	49
C_1=majority(P_0,P_1,P_2) shifted left	00011100	28
S_2=S_1 XOR C_1 XOR P_3	01110101	117
C_2=majority(S_1,C_1,P_3) shifted left	00110000	48

Both 49+28+88 and 117+48 equal 165=11×15. Majority is bitwise; it does not consume a neighboring carry.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

CH03-Q14 (continued)

2. Final carries and the missing-shift error

Position j, low to high	0	1	2	3	4	5	6	7
$S_2[j]$	1	0	1	0	1	1	1	0
$C_2[j]$	0	0	0	0	1	1	0	0
Carry-in c_j	0	0	0	0	0	1	1	1
Result bit	1	0	1	0	0	1	0	1
Carry-out c_{j+1}	0	0	0	0	1	1	1	0

Read high to low: $10100101_2=165$. Omitting both left shifts gives erroneous first-level rows 49 and 14, then erroneous second-level rows 103 and 24 from inputs 49,14,88. Their sum is 127. The bug halves the carry's weight.

CH03-Q14 (continued)

3. Sum the one-column identities

For a bit sum 0,1,2 or 3, $x_j+y_j+z_j=s_j+2k_j$. Multiply by 2^j and sum over j=0,...,W−1. This proves X+Y+Z=S+C, including position W of C. Three four-bit values 15 give $S=1111_2=15$,$C=11110_2=30$ and exact total $101101_2=45$, requiring six bits. In four-bit modular arithmetic, truncate C to $1110_2=14$ and obtain (15+14) mod 16=13=1101_2. The exact maximum is $3(2^W-1)$, which can require W+2 bits for $W\geq 2$. For W=1 the maximum is 3 and needs only two bits.

CH03-Q14 (continued)

4. Count levels before evaluating delay

Nine rows become 6,4,3,2: four levels. The stated upper-bound model gives $40+4\times 60+180+20=480$ ps. This is a model evaluation, not a measurement or a comparison against an unspecified competing circuit. Such a comparison needs its adder, wiring, load and technology data.

CH03-Q14 (continued)

5. All five independent transactions

Transaction	Accept edge	Accept time/ps	Complete edge	Complete time/ps
A	0	0	3	750
B	1	250	4	1000
C	2	500	5	1250
D	3	750	6	1500
E	4	1000	7	1750

Latency is 750 ps, initiation interval 250 ps and final completion 1750 ps. If B consumes A's product, it lacks an operand at edge 1 and must be delayed according to the actual interface and sampling constraints. The independent-transaction assumption fails; the CSA arithmetic remains valid.

References and attribution

Lin Cing, handwritten computer organization notes. Patterson and Hennessy, Computer Organization and Design, revised fourth edition, Chapter 3. CSO course slides, arithmetic.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.
https://github.com/xinhi6716/computer-science-self-study